

actions.py

```
1 from collections import deque
2 from point import Point
3 import numpy as np
4
5 def rect(buffer, start: Point, stop: Point, color, fill = None, maskBuffer = None):
6
7     x1, x2 = sorted([start.x, stop.x])
8     y1, y2 = sorted([start.y, stop.y])
9
10    buffer[y1 : y2 + 1, x1 : x2 + 1] = color
11
12    if not isinstance(maskBuffer, type(None)):
13        maskBuffer[y1 : y2 + 1, x1 : x2 + 1] = True
14
15
16 def line(buffer, start: Point, stop: Point, color, maskBuffer = None):
17
18     steps = max(np.abs(np.array((start.x - stop.x, start.y - stop.y)))) + 1
19
20     # linear interpolation
21
22     points = np.round(
23         np.linspace(
24             (start.x, start.y),
25             (stop.x, stop.y),
26             steps
27         )).astype(int)
28
29     buffer[points[:, 1], points[:, 0]] = color
30
31     if not isinstance(maskBuffer, type(None)):
32         maskBuffer[points[:, 1], points[:, 0]] = True
33
34 def ellipse(buffer, start: Point, stop: Point, color, fill: bool, maskBuffer =
None):
35     x1, x2 = sorted([start.x, stop.x])
36     y1, y2 = sorted([start.y, stop.y])
37
38     rx = (x2 - x1) // 2
39     ry = (y2 - y1) // 2
40     cx = x1 + rx
41     cy = y1 + ry
42
43     rx2 = rx * rx
44     ry2 = ry * ry
45
46     x = 0
```

```

47     y = ry
48
49     dx = 2 * ry2 * x
50     dy = 2 * rx2 * y
51
52     def plot(px, py):
53         if fill:
54             # draw horizontal spans
55             for fx in range(cx - px, cx + px + 1):
56                 buffer[cy + py, fx] = color
57                 buffer[cy - py, fx] = color
58                 if not isinstance(maskBuffer, type(None)):
59                     maskBuffer[cy + py, fx] = True
60                     maskBuffer[cy - py, fx] = True
61         else:
62             points = [
63                 (cx + px, cy + py),
64                 (cx - px, cy + py),
65                 (cx + px, cy - py),
66                 (cx - px, cy - py),
67             ]
68             for px_, py_ in points:
69                 buffer[py_, px_] = color
70                 if not isinstance(maskBuffer, type(None)):
71                     maskBuffer[py_, px_] = True
72
73     # --- Region 1 ---
74     p1 = ry2 - rx2 * ry + 0.25 * rx2
75
76     while dx < dy:
77         plot(x, y)
78
79         if p1 < 0:
80             x += 1
81             dx += 2 * ry2
82             p1 += dx + ry2
83         else:
84             x += 1
85             y -= 1
86             dx += 2 * ry2
87             dy -= 2 * rx2
88             p1 += dx - dy + ry2
89
90     # --- Region 2 ---
91     p2 = (ry2 * (x + 0.5)**2 +
92           rx2 * (y - 1)**2 -
93           rx2 * ry2)
94
95     while y >= 0:

```

```

96         plot(x, y)
97
98     if p2 > 0:
99         y -= 1
100         dy -= 2 * rx2
101         p2 += rx2 - dy
102     else:
103         y -= 1
104         x += 1
105         dx += 2 * ry2
106         dy -= 2 * rx2
107         p2 += dx - dy + rx2
108
109 def floodFill(buffer, point: Point, color):
110
111     targetColor = tuple(buffer[point.y, point.x])
112
113     # Either current color, or if over network: provided color
114     newColor = color
115
116     if targetColor == newColor:
117         return
118
119     width, height, _ = buffer.shape
120
121     def onCanvas(x, y):
122         return 0 <= x < width and 0 <= y < height
123
124     queue = deque()
125     queue.append(point)
126
127     while queue:
128         x, y = queue.popleft()
129
130         if not onCanvas(x, y):
131             continue
132
133         if not tuple(buffer[y, x]) == targetColor:
134             continue
135
136         buffer[y, x] = newColor
137
138         queue.append((x - 1, y))
139         queue.append((x + 1, y))
140         queue.append((x, y + 1))
141         queue.append((x, y - 1))

```